

UNIVERSITÀ DEGLI STUDI DELL'INSUBRIA

DIPARTIMENTO DI SCIENZE TEORICHE E APPLICATE

CORSO DI STUDIO TRIENNALE IN
INFORMATICA

Sviluppo di uno strumento di migrazione dati Infor LN

Relatore:
Prof. Massazza Paolo

Tesi di Laurea di:
Gabriele Caretti
Matricola 756564

Anno accademico:
2025/2026

Indice

1 Introduzione **1**

1.1 Contesto Aziendale 1

1.2 Obiettivi del Progetto 1

1.3 Struttura della Tesi 1

2 Stato Attuale **3**

2.1 Sistemi ERP 3

2.1.1 Evoluzione dai Sistemi Legacy al Cloud 3

2.2 Infor LN e CloudSuite Enterprise 3

2.2.1 Il Linguaggio Baan C: 3GL e 4GL 4

2.2.2 Infor Exchange e Strumenti di Import/Export 4

2.3 Migrazione Dati nei Sistemi ERP 5

2.3.1 Sfide e Problematiche Comuni 5

3 Analisi dei Requisiti **6**

3.1 Scenario di Riferimento 6

3.1.1 Il Flusso di Migrazione Esistente 6

3.2 Requisiti Funzionali 6

3.2.1 Sviluppo dell'Importer Infor Exchange 6

3.2.2 Sistema di Estendibilità con Funzioni Ad-hoc 7

3.2.3 Integrazione con lo Strumento Documentale 7

3.3 Requisiti Non Funzionali 7

3.3.1 Manutenibilità ed Estensibilità 7

3.3.2 Sicurezza e Data Integrity in Multitenancy 7

3.4 Modello dei Dati (Entità della Migrazione) 8

3.4.1 Analisi delle Strutture Dati Coinvolte 8

4 Architettura **9**

4.1 Panoramica dell'Architettura di Sistema 9

4.2 Progettazione del Software 9

4.2.1 Design dell'Importer e del Parser dei Dati 9

4.2.2 Motore di Generazione del Codice 9
4.2.3 Modulo di Integrazione Ad-hoc e Callbacks 10
4.3 Scelte Tecnologiche 10
4.3.1 Linguaggi e Framework 10
4.3.2 Librerie di Supporto e Strumenti di Build 10

5 Implementazione **11**
5.1 Sviluppo dell'Importer Infor Exchange 11
5.1.1 Gestione dei Formati e Parsing 11
5.1.2 Ricostruzione delle Definizioni dei File da Migrare 11
5.2 Implementazione del Generatore di Codice 12
5.2.1 Template Engine e Creazione Dinamica 12
5.2.2 Supporto alle Funzioni di Trasformazione Ad-hoc 12
5.3 Interfacce Utente (Sessioni) 12
5.4 Integrazione Documentale 13
5.4.1 Esportazione delle Specifiche di Migrazione 13

6 Testing e Validazione **14**
6.1 Strategia di Testing 14
6.1.1 Test Unitari sui Moduli di Parsing 14
6.1.2 Test di Integrazione del Flusso di Migrazione 14
6.2 Verifica dell'Integrità dei Dati 15
6.2.1 Matching tra Dati Sorgente (Legacy) e Destinazione (Cloud) 15
6.3 Metriche e Miglioramenti Prestazionali 15

7 Conclusioni **16**
7.1 Obiettivi Raggiunti e Risultati Ottenuti 16
7.2 Sfide Incontrate 16
7.3 Sviluppi Futuri 17

Bibliografia **18**

Capitolo 1

Introduzione

1.1 Contesto Aziendale

Il progetto di tesi si propone di sviluppare uno strumento di migrazione dati per Infor LN [3], un sistema ERP (Enterprise Resource Planning) industriale utilizzato a livello internazionale.

L'obiettivo è agevolare il processo di migrazione da sistemi legacy a Infor LN, garantendo l'integrità e la coerenza dei dati in ambienti multitenant [1, 2].

1.2 Obiettivi del Progetto

Il lavoro si concentrerà sull'analisi dei requisiti, la progettazione dell'architettura del software, la stesura della documentazione e lo sviluppo di un prototipo funzionante.

In particolare, lo strumento sarà in grado di generare dinamicamente codice, estendibile con funzioni ad hoc, per migrare dati su sistemi Cloud Multitenant, con un focus sulla Infor Cloudsuite Enterprise.

Inoltre, è richiesto lo sviluppo di un importer capace di interpretare il formato di import/export di Infor Exchange al fine di ricreare le definizioni dei file da migrare, integrandolo con lo strumento aziendale già esistente per la produzione di specifiche di migrazione documentate.

1.3 Struttura della Tesi

L'elaborato si articola nel seguente modo:

- **Capitolo 2**: stato attuale degli ERP moderni, architetture cloud multi-tenant e sfide di migrazione dati.
- **Capitolo 3**: analisi dello scenario di partenza e dei requisiti funzionali e non funzionali.
- **Capitolo 4**: architettura del sistema, design e scelte tecnologiche (come Importer e Generatore codice).
- **Capitolo 5**: descrizione dell'implementazione concreta del progetto.
- **Capitolo 6**: testing, inclusi elementi come la validazione dell'integrità dei dati.
- **Capitolo 7**: conclusioni, traguardi raggiunti e possibili sviluppi futuri.

Capitolo 2

Stato Attuale

2.1 Sistemi ERP

I sistemi ERP (Enterprise Resource Planning) rappresentano un elemento essenziale del sistema gestionale aziendale [4], consentendo l'integrazione dei processi di business principali di un'azienda (vendite, acquisti, gestione inventario, contabilità).

2.1.1 Evoluzione dai Sistemi Legacy al Cloud

Nel tempo, i sistemi ERP si sono evoluti partendo da architetture on-premise legacy, spesso caratterizzate da un elevato grado di personalizzazione (customization, in Infor LN definiti come tx) e importanti costi di mantenimento. Tuttavia, oggi si sta assistendo ad uno spostamento verso nuove soluzioni Cloud.

Il modello Cloud Multitenant rappresenta l'architettura degli ERP odierni [1]. Ove, una singola istanza del software e infrastruttura servono più clienti (definiti come tenant). Questo approccio offre scalabilità, aggiornamenti continui e ottimizzazione dei costi, ma anche diverse sfide nell'ambito della gestione e segregazione dei dati, nonché della sicurezza, richiedendo che ogni processo di migrazione operi secondo specifici vincoli di integrità [5].

2.2 Infor LN e CloudSuite Enterprise

Infor LN, nato con il nome di Baan ERP, è uno dei principali ERP nel settore manifatturiero [3] a livello internazionale. Viene integrato all'interno della Infor CloudSuite Enterprise, il quale offre funzionalità per la gestione della supply chain e produzione [2].

Infor LN è basato su un'architettura che separa il framework tecnologico (gli Infor Tools) dalla logica di business. Ogni applicazione è composta da diversi oggetti:

- **Tabelle (Tables):** I dati fisici, definiti tramite Data Dictionary.
- **Sessioni (Sessions):** Le interfacce utente (da qui definite come Sessioni) attraverso cui gli operatori accedono, visualizzano e interagiscono con i dati.
- **DAL (Data Access Layer):** L'elemento che si occupa di garantire l'integrità dei dati e delle tabelle, indipendentemente dalle Sessioni.

2.2.1 Linguaggio Baan C: 3GL e 4GL

Per lo sviluppo e personalizzazione, Infor LN utilizza un linguaggio proprietario chiamato **Baan C**.

Esso si divide a sua volta in:

- **4GL (Fourth-Generation Language):** La versione più recente e moderna. Infor LN Tools genera automaticamente operazioni standard di Sessione e database (CRUD). In 4GL non è necessario scrivere codice per gestire il salvataggio o il caricamento di un record, ma ci si concentra sulla gestione degli *eventi* legati ai componenti della Sessione o del database (tramite DAL).
- **3GL (Third-Generation Language):** Precedente a 4GL, e totalmente integrato con esso, viene utilizzato per operazioni come gestione di logiche di business complesse. Permette un approccio più procedurale e a basso livello per implementare funzioni matematiche, algoritmi personalizzati, DLL (Dynamic Link Libraries) proprietarie, funzioni e routine di manipolazione stringhe o file.

2.2.2 Infor Exchange e Strumenti di Import/Export

Per lo scambio di informazioni, Infor fornisce un modulo denominato *Exchange*. Si tratta di una piattaforma per l'importazione e l'esportazione di file (es. CSV, XML, testo a larghezza fissa) mappando righe e colonne sui *Datafiles* e sui campi del database di LN.

Spesso sono necessarie delle personalizzazioni delle procedure di migrazione tramite Baan C, che quindi risulta essenziale quando le regole di mappatura superano la complessità gestibile nativamente dall'Exchange.

2.3 Migrazione Dati nei Sistemi ERP

2.3.1 Sfide e Problematiche Comuni

La migrazione da un sistema legacy a un nuovo ERP cloud, richiede un'attenta fase di analisi per evitare perdite o disallineamenti di informazioni storiche e master data.

Attualmente, molte migrazioni sono per la maggior parte gestite manualmente, con l'integrazioni di script scritti ad hoc, con tutti i rischi che ne conseguono. Per questo, nasce l'idea del Tool di Migrazione (*Migration Tool*) oggetto di questa tesi.

Per la migrazione verso il Cloud, sono necessari tool robusti in grado di lavorare sia con logiche batch sia che con API e servizi web (*webservices* o WS in breve), adattandosi alle regole di business e garantendo la qualità di ogni singolo dato prima ancora che venga scritto nel database del nuovo tenant.

Capitolo 3

Analisi dei Requisiti

3.1 Scenario di Riferimento

3.1.1 Flusso di Migrazione Esistente

Negli ambienti legacy, il processo di migrazione dei dati [5] si basa su procedure manuali e script ad hoc.

L'operatore (noi) è costretto a sviluppare script personalizzati per ogni singola tabella, sessioni, e ogni elemento di un gestionale, con lunghi tempi di realizzazione e rischi inerenti alla trascrizione manuale delle logiche di mapping.

Inoltre, cambiamenti da parte di Infor e nuovi standard più restrittivi hanno reso le operazioni manuali molto più complesse [4]. Emerge quindi la necessità aziendale di innovare, ottimizzare e automatizzare il processo con un nuovo strumento, ossia l'integrazione di un *Migration Tool*.

3.2 Requisiti Funzionali

I requisiti funzionali definiscono i comportamenti attesi, ossia che cosa dovrebbe fare il nuovo strumento, così come stabiliti con gli stakeholder.

3.2.1 Sviluppo dell'Importer Infor Exchange

Il sistema dovrà supportare l'input di file e strutture generate dallo strumento preesistente nativo di Infor Exchange.

Tali file presentano specifici layout (come formati testuali ASCII o delimitati) che dovranno essere interpretati per popolare il dizionario `txamg110` (Datafiles) e `txamg111` (DataFile Fields).

Il punto cruciale sarà lo strumento generatore di codice. Partendo dalle tabelle di mapping (es. `txamg203`), lo strumento dovrà generare script di elaborazione e conversione, senza l'intervento manuale del programmatore.

3.2.2 Sistema di Estendibilità con Funzioni Ad-hoc

Nonostante l'automazione, alcune regole di business legacy non potranno essere legate a semplici assegnazioni.

Viene quindi la necessità di una funzionalità di callback per permettere, ad opera dello sviluppatore, l'inserimento di script personalizzati che verranno inseriti nel processo principale durante l'esecuzione batch (`txamg3200`).

3.2.3 Integrazione con lo Strumento Documentale

Per garantire l'allineamento con gli standard del progetto, il *Migration Tool* dovrà interfacciarsi con la documentazione aziendale, permettendo la produzione ed esportazione di specifiche tecniche di migrazione, mappature, eccezioni e log generati in fase di esecuzione (necessari per debugging).

3.3 Requisiti Non Funzionali

I requisiti non funzionali gestiscono i vincoli qualitativi e infrastrutturali del software.

3.3.1 Manutenibilità ed Estensibilità

Il *Migration Tool* dovrà essere interamente sviluppato in linguaggio Baan C rispettando il framework Infor Tools, così da garantirne la compatibilità con future release dell'ERP e manutenibilità semplificata per il comparto IT aziendale.

3.3.2 Sicurezza e Data Integrity in Multitenancy

Dato che il sistema tratterà dati di tipo core (come informazioni private o riservate) dell'azienda cliente in contesti Cloud, la sicurezza e la segregazione (isolamento) dei dati per i *Tenant* costituiscono priorità assolute e fondamentali.

Inoltre, il caricamento dovrà ottimizzare l'uso della memoria e delle sessioni, prediligendo una elaborazione a blocchi al fine di minimizzare i tempi morti nati a causa della latenza di rete verso i server CloudSuite remoti.

3.4 Modello dei Dati

3.4.1 Analisi Strutture Dati Coinvolte

Per supportare in modo efficace il processo di migrazione, è stata definita un'architettura dati, basata su package *tx* (*custom*).

Il modello dati è suddiviso in diverse entità:

- **Parametrizzazione:**: tramite tabelle di configurazione come `txamg000` (AlteaIN Migration Tool Parameters).
- **Gestione dei Datafile:** tabelle come `txamg110` (Datafiles) e `txamg111` (DataFile Fields) permettono di definire le strutture dei file originali da importare.
- **Elaborazione Batch:** la configurazione e l'esecuzione tramite batch è regolata da tabelle come `txamg200` (Batch), `txamg201` (Batch Sequences) e `txamg202` (Target Tables), insieme ad altre tabelle dedicate al mapping dei campi (`txamg203`).
- **Tracciamento Esecuzioni:** lo stato e l'esito delle operazioni, oltre all'eventuale presenza di errori, vengono memorizzati all'interno delle tabelle `txamg300`, `txamg301`, `txamg302` e `txamg303` (Runs Errors).

I Domini includono sia tipi strettamente descrittivi (es. `txamg.cdom`, `txamg.fdnm`) sia tipi enumerativi per la validazione e il workflow (es. `txamg.migt` o `txamg.mapl`).

Esempi di Domini essenziali sono quelli relativi ai formati ASCII (`txamg.fast`), alle regole di mapping (`txamg.mapl`) o al tracciamento dei file (`txamg.dfil`).

Questi elementi compongono le fondamenta sulla quale si poggia l'integrazione per la generazione del codice di migrazione e dell'importer Infor Exchange.

Capitolo 4

Architettura

4.1 Panoramica dell'Architettura di Sistema

L'architettura del Migration Tool è stata definita per essere modulare e scalabile. Alla base del sistema vi è una separazione netta tra il modulo di input (Importer), il motore per l'elaborazione dei metadati, e il Generatore di Codice. Questo approccio garantisce una separazione tra il layout dei dati preesistenti e la struttura del database Infor CloudSuite.

4.2 Progettazione del Software

4.2.1 Design dell'Importer e del Parser dei Dati

L'Importer funge da punto d'ingresso per i dati ed è progettato per interpretare il formato proprietario di Infor Exchange, analizzando file di esportazione testuali o strutturati. Estraendo i metadati dai record, l'Importer popola i dizionari *Datafiles* (`txamg110`) e i relativi *Fields* (`txamg111`).

Questo design del caricamento rende possibile supportare sia architetture multitenant cloud, sia installazioni legacy on-premise.

4.2.2 Motore di Generazione del Codice

Il nucleo computazionale è rappresentato dal motore di generazione. Basato su librerie compilate (DLL come `txamgdll0012` e `txamgdll0013`), il modulo interpreta il dizionario di mapping (definito in `txamg203` o *MappedFields*) e traduce i legami e le regole di trasformazione in script nativi Baan C.

Questo meccanismo previene di dover scrivere manualmente codice, garantendo il rispetto dei framework CloudSuite: il codice viene prima generato, poi compilato via `bic_compiler`, ed infine agganciato ai *Batch Runs*.

4.2.3 Modulo di Integrazione Ad-hoc e Callbacks

Mentre buona parte dei flussi vengono standardizzati automaticamente, alcune regole di business richiedono sviluppi specifici a causa della loro complessità. Vengono quindi esposte a livello architetturale delle interfacce *callback*.

In appositi *module script*, gli sviluppatori possono iniettare blocchi di codice chiamati asincronamente dall'engine. Questi script personalizzati usufruiscono degli stessi vantaggi transazionali dell'utility principale.

4.3 Scelte Tecnologiche

4.3.1 Linguaggi e Framework

L'intero ecosistema è sviluppato sfruttando l'Infor Tools Framework, programmando in **Baan C (sia 3GL che 4GL)**. In parte sono state adottate le *DAL (Data Access Layer)*, per il restante, sono state utilizzati approcci più classici e a basso livello.

4.3.2 Librerie di Supporto e Strumenti di Build

Tutto il coordinamento della configurazione, logging (`txamgdll0011`) e le *string utility* si basano sulle librerie del sistema ERP (DLL native Baan e tool come il `ttdll string`).

Le build non prevedono toolchain esterni complessi (come Maven o MSBuild), ma si appoggiano direttamente al toolset interno dell'ERP, il quale gestisce i legami VRC (Version Release Customer) multitenant per garantirne l'incapsulamento del codice generato al corretto Tenant.

Capitolo 5

Implementazione

5.1 Sviluppo dell'Importer Infor Exchange

5.1.1 Gestione dei Formati e Parsing

L'importer è stato sviluppato come un modulo d'input che si interpone tra i file d'ingresso (prodotti dai vecchi server o estratti tramite tool legacy, a volte anche Access) e le tabelle di definizione del nuovo sistema.

La libreria dedicata alla gestione dei metadata (`txamgdll0011`) prende in carico l'interpretazione dei metadati (quali formati delimitati o logiche ASCII a lunghezza fissa) eseguendo conversioni su base parametrica secondo l'enumerato `txamg.fast`.

Ogni riga recuperata viene controllata per garantire la conformità delle codifiche testo e i troncamenti (*trim*) o padding dei campi vuoti.

5.1.2 Ricostruzione delle Definizioni dei File da Migrare

L'obiettivo secondario dell'importer è auto-compilare il setup dello stesso *Migration Tool*. Utilizzando le definizioni originarie dei file (le tabelle legacy), il programma crea dinamicamente o aggiorna una mappatura interna dei *Data Files* attraverso Sessioni back-end come `txamg1110m000`.

Le descrizioni, numerazioni dei campi (`txamg.fdno`) ed i tipi di dominio correlati (`txamg.atyp`) vengono riversati nella tabella di supporto `txamg111` e sono quindi disponibili e precompilati per le successive attività di data-mapping manuale verso le tabelle target CloudSuite.

5.2 Implementazione del Generatore di Codice

5.2.1 Template Engine e Creazione Dinamica

La generazione dinamica del codice è gestita mediante apposite librerie DLL (*Dynamic Link Library*) fornite nel sistema Infor, quali `txamgdll0012` (Script Generator - Load dll) e `txamgdll0013` (Script Generator - Process dll).

Il motore costruisce letteralmente il codice sorgente Baan C riga per riga, definendo le firme delle funzioni (ad esempio `function extern long process.import.file...()`) e le variabili necessarie. Per farlo conformemente all'infrastruttura di Infor LN, il generatore interroga direttamente il Data Dictionary del sistema tramite le tabelle di sistema `ttadv` (in particolare `tttadv420` per le definizioni delle tabelle, `tttadv422` per i campi e `tttadv400` per i domini). In questo modo, tipi di dato, lunghezze e array sono dedotti in automatico dal dizionario applicativo.

Tali librerie servono ad analizzare la configurazione definita nelle tabelle di mapping (ad esempio `txamg203`) per generare script eseguibili, riducendo drasticamente il tempo manuale richiesto per configurare le regole di trasformazione standard e garantendo che il codice finale aderisca alla logica di business (*business logic*).

5.2.2 Supporto alle Funzioni di Trasformazione Ad-hoc

Oltre alle regole predefinite, l'interprete ammette la dichiarazione e l'aggancio di funzioni custom. Durante il processo, eventuali routine di validazione o pulizia del dato possono essere inoltrate ad altre interfacce di esecuzione (come `txamg3200` - Execute Batch Runs).

L'ausilio del componente `txamgdll0011` è importante in quanto si occupa del management dei metadati e dei log contenenti eventuali errori (Error Logging).

5.3 Interfacce Utente (Sessioni)

L'interazione con l'utente avviene attraverso un set completo di sessioni dedicate, denominate *m-sessions* o *s-sessions* a seconda della loro finalità operativa o di supporto.

Si evidenziano, per esempio:

- **Setup iniziale:** la Sessione `txamg0100m000` permette la parametrizzazione globale dello strumento.
- **Definizione Formati:** le Sessioni relative a `txamg1110` gestiscono il popolamento dei formati dati elaborati.
- **Mapping Dati:** la definizione di conversione logica e associazioni campo-campo tra origine e destinazione avviene nelle interfacce di visualizzazione delle tabelle *target* (`txamg2102m000`) e *fields mapping* (`txamg2103m000`).

5.4 Integrazione Documentale

5.4.1 Esportazione delle Specifiche di Migrazione

L'integrazione formale del modulo d'implementazione sfocia nella sua interconnessione con lo strumento aziendale adibito alla documentazione del progetto.

Prima della configurazione ed eventuale esecuzione esplorativa del batch (le sessioni `txamg2100` e `txamg3200`), l'*utility* estrae il risultato dei mapping salvato nel `txamg203` (Tabella Mappings), formattandolo utilizzando layout predefiniti.

L'esportazione documenta in PDF ogni singolo dominio trasferito, documentando sia i campi copiati senza modifiche (*one-to-one*) (in accordo con `txamg.mapl`) sia quei casi particolari per cui è stato inserito del codice personalizzato, semplificando una possibile manutenzione logica dell'architettura in futuro.

Capitolo 6

Testing e Validazione

6.1 Strategia di Testing

Il testing di un software per la migrazione dei dati è un aspetto fondamentale. Data la sensibilità delle logiche del *Data Dictionary* di Infor CloudSuite in ambienti Cloud Multitenant, ogni anomalia nel record di inserimento avrebbe gravi conseguenze nel sistema target.

Inoltre, considerando che il cuore del sistema è un generatore dinamico di codice Baan C, le procedure di collaudo hanno dovuto coprire non solo l'importer dei dati, ma anche la correttezza sintattica e logica degli script generati a runtime dalle DLL `txamgdll0012` e `txamgdll0013`.

6.1.1 Unit Testing sui Moduli di Parsing

I moduli di base (DLL quali `txamgdll0011`) per l'estrazione e la gestione dei metadata (*metadata management*) degli Infor Exchange sono stati collaudati su porzioni specifiche di file.

Verificando ogni singola colonna (`txamg1111`), la validazione include il corretto casting di tipi enumerati, decimali a virgola mobile e codifiche caratteri ASCII, simulando una diversità di input.

6.1.2 Test di Integrazione del Flusso di Migrazione

L'integrazione end-to-end è validata eseguendo flussi di test governati dalle logiche di batch (`txamg2100` e `txamg3200`). Durante questi test, il sistema genera gli script di elaborazione e li sottopone al compilatore interno `bic_compiler`. Se la generazione avviene a regola d'arte, lo script compilato viene agganciato al batch e lanciato.

I *Runs Errors* offrono un riscontro immediato di blocchi logici o semantici. Sono stati eseguiti scenari in cui la validazione è forzata per respingere record privi di riferimenti chiave, verificando che la linea incriminata fosse elaborata senza però interromperne il caricamento dell'intero *Run*.

6.2 Verifica dell'Integrità dei Dati

6.2.1 Matching tra Dati Sorgente (Legacy) e Destinazione (Cloud)

Ogni file migrato è dotato di un tracciamento specifico (`txamg301`, `txamg302`). Il *matching* prevede che vengano comparati il numero dei record letti, processati ed eventualmente respinti rispetto alla fonte.

I report prodotti (tramite moduli ad hoc come `txamgdll0011` per il log d'errore) forniscono la garanzia numerica dell'operazione.

Capitolo 7

Conclusioni

7.1 Obiettivi Raggiunti e Risultati Ottenuti

Nel percorso di questa tesi, è stato analizzato, implementato e documentato un completo *Migration Tool* (pacchetto aziendale `txamg`) per l'ambiente Infor LN. C'erano requisiti particolarmente sfidanti dietro a questo progetto, nati per la maggior parte dalla natura multitenant dei nuovi ambienti CloudSuite, che sono stati affrontati offrendo un approccio fortemente parametrico e automatizzato.

Lo sviluppo di librerie capaci di generare dinamicamente codice leggendo il dizionario (come `txamgdll0012` e `txamgdll0013`) e un importer capace di decifrare nativamente i formati Infor Exchange, ha permesso di centrare in pieno l'obiettivo aziendale richiesto. In questo modo si sono potuti garantire ottimi risultati in termini di affidabilità e performance, alleggerendo notevolmente i consulenti dallo snervante e ripetitivo compito della trascrizione manuale, che spesso si traduceva in facili errori.

7.2 Sfide Incontrate

Le principali difficoltà che hanno accompagnato tutto il periodo di progetto sono nate dal dover imporre un mapping rigoroso, lavorando in gran parte su un sistema chiuso (proprietario Infor), in cui l'utilizzo esclusivo dei linguaggi Baan C (3GL e 4GL) ha rappresentato il vincolo tecnico ineliminabile.

Lavorando in questo ecosistema ristretto, avere la possibilità di interfacciarsi con il compilatore interno (e validare il codice target a runtime su ogni specifico tenant) ha fatto un'enorme differenza dal punto di vista dell'analisi e debug. Tuttavia, standardizzare formati eterogenei del mondo legacy ha imposto la

creazione di strumenti di parsing e configurazione di *Runs* batch piuttosto complessi per il modulo `txamg200`.

Durante lo fase di studio e sviluppo, è stato necessario affrontare una fase di training sia base che avanzata dei tool Infor, quali struttura, nomenclatura, linguaggi, logica di funzionamento, architettura come *middleware ION*, BOD, API e *Web Services* con formati XML legacy come SOAP (non REST), in tempi molto brevi rispetto a quanto normalmente previsto (diversi mesi), al fine di completare il progetto in tempo e completare questa tesi.

Si fa anche notare la natura molto restrittiva dell'ERP Infor, che non permette l'accesso nemmeno ad amministratori di sistema a parti di API o funzionalità che semplificherebbero lo sviluppo, oltre alla mancata documentazione delle stesse in molti casi a causa di transizioni a più alto livello aziendale, lasciando allo sviluppatore finale (noi) "scoprire" ed esplorare tramite trial-and-error delle soluzioni, spesso non standard, ove possibile. Questo ha dato origine molti tempi morti, complicando il contesto generale

7.3 Sviluppi Futuri

Nonostante lo standard d'eccellenza raggiunto, si ritiene possibile evolvere ulteriormente il tool in diverse direzioni.

In primo luogo, lo schema architetturale per le estensioni *Custom* potrebbe venire affiancato da procedure di *Machine Learning* (addestrate sui mapping legacy) in grado di suggerire i mapping e i moduli di conversione per grandi volumi di tabelle non ancora definite (senza l'input umano in interfacce e Sessioni `txamg2103`). Inoltre, la visualizzazione delle specifiche tramite esportazione a modelli UML o grafi digitali, se automatizzata nativamente dal framework, fornirebbe reportistica in *real-time* durante la fase di studio del cliente prima dell'operazione di caricamento, riducendo di fatto tempi e costi e supportando i consulenti AlteaIN, funzionali e Infor verso transizioni Cloud più semplici.

Bibliografia

[1] Y. Guo et al. «Multi-tenant cloud computing architecture: A review». In: *Journal of Ambient Intelligence and Humanized Computing* (2020) (cit. alle pp. 1, 3).

[2] *Infor Cloudsuite Enterprise*. Accessed: 2026-04-16. 2026. URL: `https://www.infor.com/products/cloudsuite-enterprise` (cit. alle pp. 1, 3).

[3] *Infor LN - Cloud ERP for Manufacturing*. Accessed: 2026-04-16. 2026. URL: `https://www.infor.com/products/ln` (cit. alle pp. 1, 3).

[4] B. Johansson e F. Sudzina. «Challenges in Data Migration to ERP Systems». In: *Enterprise Information Systems* (2008) (cit. alle pp. 3, 6).

[5] Johny Morris. *Practical Data Migration*. BCS, The Chartered Institute for IT, 2012 (cit. alle pp. 3, 6).